



Banco de Dados Técnicas de Recuperação de Banco de Dados

Marcel Oliveira

1



Nas Aulas Anteriores...

- Diferentes causas de falhas
 - Quedas de sistemas e erros de transações
- Conceitos usados na recuperação
 - Histórico do sistema
 - Pontos de efetivação

2



Nesta Aula...

- Técnicas de Recuperação de BD
 - Restauração Típica
 - Categorias de Algoritmos de Recuperação
 - Conceitos de Recuperação
 - Gravação adiantada de log
 - Atualizações *in loco* x sombra
 - Processo de *rollback*
 - Efeitos de transações incompletas

3



Nesta Aula...

- Técnicas de Recuperação de BD
 - Técnicas de recuperação com atualização adiada (NO-UNDO/REDO)
 - Técnicas de recuperação com atualização imediata (UNDO/REDO e UNDO/NO-REDO)
 - Técnicas de recuperação com sombra (NO-UNDO/NO-REDO)

4



Nesta Aula...

- Técnicas de Recuperação de BD
 - Recuperação em diversos BDs
 - Recuperação de falhas catastróficas

5



Esquema de Recuperação

- Ocorre uma falha
- Recuperação para estado consistente do BD
- Informações para recuperação estão no log

6



Esquema de Recuperação

- Estratégia Típica
 - Falha Catastrófica (recuperação com backup)
 - Restaurar cópia anterior do BD
 - Aplicar atualizações (baseado no log) até o momento da falha
 - Falha não catastrófica
 - Reverter mudanças desfazendo operações
 - Também pode ser necessário refazer operações
 - Por enquanto, focamos na segunda

7



Técnicas Principais de Recuperação de Falhas

- Atualização Adiada
- Atualização Imediata

8



Técnicas Principais de Recuperação de Falhas

- Atualização Adiada
 - Atualiza o BD apenas após a efetivação da transação (*commit*)
 - Antes da efetivação, todas as transações atualizam seus *buffers*
 - Durante efetivação, transações registram atualizações no log e depois no BD
 - Falha antes da efetivação (NO-UNDO)
 - Sem alteração no BD
 - Não necessita de UNDO
 - Falha durante efetivação (REDO)
 - Pode não ter persistido no BD
 - Necessita REDO

9



Técnicas Principais de Recuperação de Falhas

- Atualização Imediata
 - Atualiza o BD durante execução da transação
 - Registra atualizações no log
 - Falha antes da efetivação (UNDO-REDO)
 - BD foi alterado, efeitos da transação precisam ser revertidos
 - Necessita de UNDO
 - Necessita de REDO para refazer outros efeitos
 - Mais frequentemente usada
 - Variação
 - UNDO/NO-REDO

10



Conceitos

Caching (Bufferização)

- Atualização de páginas de discos (blocos) com itens de dados nos buffers da memória principal
- Manipulação pelo SGBD de páginas em discos
 - Chamadas de rotinas de baixo nível do SO
- Utiliza um catálogo
 - Que itens do BD estão no Buffer

11



Conceitos

Caching (Bufferização)

- Ações em um item de dados
 - Localização do item (disco ou cache)
 - Pode necessitar de substituição no buffer
 - Estratégias *least recent used* e FIFO podem ser usadas
 - Utilização de *dirty bit* que indica alteração no item
 - Utilização de *pin-unpin* (*pinned* se não puder ser escrita no disco)

12



Conceitos ***Caching (Bufferização)***

- Estratégias
 - Atualizando no local
 - Regrava o buffer no mesmo local de origem no disco
 - *Shadowing*
 - Regrava o buffer em uma localização diferente
 - Várias versões do mesmo item

13



Conceitos ***Logging Write-Ahead***

- Usando Atualizando no Local precisamos de log de recuperação
- Mecanismo de recuperação deve garantir que imagem anterior esteja registrada no log e seja transferida para o disco antes que ela seja sobrescrita pela imagem posterior no disco

14



Conceitos

Logging Write-Ahead

- Dois tipos de informação no log para operações de escrita
 - REDO inclui o novo valor para refazer operações
 - UNDO inclui o valor antigo para desfazer operações
 - Necessárias para REDO e UNDO
 - Algoritmos UNDO/REDO usam ambas as informações

15



Conceitos

Logging Write-Ahead

- Cache contém blocos de dados, blocos de índice e blocos de log
- Atualização gera registro de log no último bloco da cache
- *Write-ahead (escrita adiantada)*
 - Blocos de log de uma atualização são gravados em disco antes do bloco de dados atualizado correspondente

16



Conceitos

(não)-Roubada e (não)-Forçada

- Quando uma página de BD pode ser gravada em disco a partir da cache:
 - Roubada vs Não-roubada
 - Roubada
 - página atualizada por transação pode ser gravada antes da efetivação da transação
 - Não-roubada
 - caso contrário

17



Conceitos

(não)-Roubada e (não)-Forçada

- Quando uma página de BD pode ser gravada em disco a partir da cache:
 - Forçada vs Não-forçada
 - Forçada
 - todas as páginas atualizadas por uma transação são imediatamente escritas no disco quando a transação se efetiva
 - Não-forçada
 - caso contrário

18



Conceitos

(não)-Roubada e (não)-Forçada

- Técnicas baseadas da Atualização adiada usam não-roubada
- Normalmente é usado roubada + não-forçada
 - Roubada requer pouco espaço de buffer
 - Não-forçada permite que uma página permaneça no buffer após efetivação da transação
 - Reduz I/O caso página seja bastante atualizada

19



Conceitos

Outras Informações

- Lista de transações
 - Transações ativas
 - Transações efetivadas e abortadas desde o último checkpoint

20



Checkpoint

- Marca no log o momento em que o SGBD grava no disco todos os buffers modificados
 - Operações efetivadas antes de um checkpoint existente não precisam ter suas escritas refeitas
- Frequência dos checkpoints
 - Baseado em tempo
 - Baseado em número de transações

21



Checkpoint

- Submissão de checkpoint
 - Suspende execução de transações
 - Forçar gravação no disco dos buffers alterados
 - Escrever registro de checkpoint no log e forçar gravação do log em disco
 - Pode incluir informações adicionais (lista de transações ativas) para facilitar o desfazer
 - Reativar a execução das transações

22



Checkpoint

- Forçar gravação de registro pode atrasar processamento de transações
- Uso de fuzzy checkpoint reduz esse overhead
 - Reativa transações assim que registro de checkpoint for escrito
 - Não espera gravação dos buffers
 - Checkpoint anterior é válido até conclusão da escrita dos buffers
 - Após isso, novo checkpoint é válido

23



Rollback

- Reverter alterações no BD feitos pelas transações pode se fazer necessário
- Entradas UNDO no log são usadas com esse propósito
- Se T for revertida, qualquer transação que leu um item escrito por T deverá ser revertida e assim por diante (reversão em cascata)

24



Rollback

- Reversão em cascata tem alto custo
 - Na prática, mecanismos de reversão são implementados de modo a evitar reversões em cascata

25



Reversão em Cascata

(a)

T_1	T_2	T_3
ler_item(A)	ler_item(B)	ler_item(C)
ler_item(D)	escrever_item(B)	escrever_item(B)
escrever_item(D)	ler_item(D)	ler_item(A)
	escrever_item(D)	escrever_item(A)

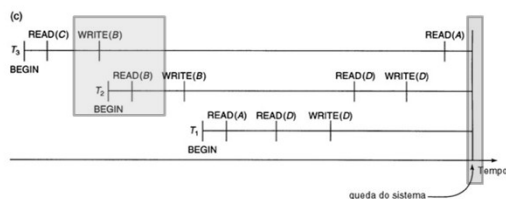
(b)

	A	B	C	D
[start_transaction, T_1]	30	15	40	20
[ler_item, T_1 , C]				
[escrever_item, T_1 , B, 15, 12]		12		
[start_transaction, T_2]				
[ler_item, T_2 , B]				
[escrever_item, T_2 , B, 12, 18]		18		
[start_transaction, T_3]				
[ler_item, T_3 , A]				
[ler_item, T_3 , D]				
[escrever_item, T_3 , D, 20, 25]				25
[ler_item, T_3 , D]				
[escrever_item, T_3 , D, 25, 26]				26
[ler_item, T_3 , A]				

← queda do sistema

* T_1 é revertida porque não alcançou seu ponto de efetivação.

** T_2 é revertida porque lê o valor do item B escrito por T_1 .



26



Reversão em Cascata

- Na prática
 - Métodos de recuperação garantem planos de execução livres de reversão em cascata
 - Log não precisa registrar leitura de item pois estes são usados apenas para reversão em cascata

27



Técnicas Baseadas na Atualização Adiada

- Posterga atualização no BD até a transação chegar ao ponto de efetivação
 - Uso do log e do buffer
- Simplifica a recuperação
- Possível na prática apenas para transações curtas e com poucas alterações de itens

28



Técnicas Baseadas na Atualização Adiada

- Protocolo Típico
 - NO-UNDO / REDO
 - Transação não pode escrever em disco até alcançar o seu ponto de efetivação
 - Transação não alcança ponto de efetivação até que todas as operações de atualização estejam registradas no log e o log seja gravado em disco

29



Técnicas Baseadas na Atualização Adiada

- Protocolo Típico
 - Log usado para refazer (REDO) operações que não foram colocados em disco mas estão no log
- Controle de concorrência pode afetar processo de recuperação
 - Recuperação em ambientes mono-usuários
 - Recuperação em ambientes multi-usuários

30



Atualização Adiada Mono-usuários

- Algoritmo RDU_S
 - *Recovery Using Deferred Update in a Single-user Environment*
 - Usa REDO para refazer operações de escrita de itens da seguinte maneira
 - Duas listas de transações
 - Efetivadas e ativas
 - REDO de todas as operações de escrita de item das transações efetivadas no log (escrita no novo valor da escrita no log)
 - Reiniciar transações ativas

31



Atualização Adiada Mono-usuários

- Considerações
 - REDO deve ser idempotente
 - Apenas transações da lista de transações ativas não terão afetado o BD

32



Atualização Adiada Mono-usuários

- Exemplo

(a)

T_1	T_2
ler_item(A)	ler_item(B)
ler_item(D)	escrever_item(B)
escrever_item(D)	ler_item(D)
	escrever_item(D)

(b)

[inicia_transacao, T_1]	REDO
[escrever_item, t_1 , D, 20]	
[confirma, T_1]	
[inicia_transacao, T_2]	
[ler_item, T_2 , B, 10]	
[escrever_item, T_2 , D, 25]	← queda do sistema

As operações [escrever_item, ...] de T_1 são refeitas.

As entradas no log de T_2 são ignoradas pelo processo de recuperação.

33



Atualização Adiada Multi-usuários

- Recuperação depende de protocolo de controle de concorrência
 - Quanto maior o grau de concorrência, maior o tempo de recuperação

34



Atualização Adiada Multi-usuários

- Exemplo
 - Controle de concorrência com bloqueio em duas fases
 - Bloqueios a itens permanecem até o ponto de efetivação
 - Planos de execução restritos e serializáveis
 - Checkpoints foram registrados no log
 - Recuperação RDU_M (*Recovery Using Deferred Update in a Multiuser Environment*)

35



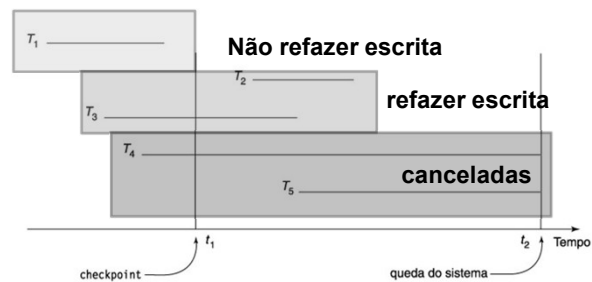
Atualização Adiada Multi-usuários

- Recuperação RDU_M (*Recovery Using Deferred Update in a Multiuser Environment*)
 - Usa REDO para refazer operações de escrita de itens da seguinte maneira
 - Duas listas de transações
 - Efetivadas desde o último checkpoint e ativas
 - REDO de todas as operações de escrita de item das transações efetivadas no log (escrita no novo valor da escrita no log)
 - Transações ativas são canceladas e devem ser re-submetidas

36



Atualização Adiada Multi-usuários



37



Atualização Adiada Multi-usuários

- Otimização
 - Analisando o log
 - Se um item de dados foi atualizado por mais de uma transação desde o último checkpoint, só precisamos refazer a última atualização
 - Iniciamos pelo fim do log
 - Mantemos uma lista de refeitos
 - Aplicamos o algoritmo anterior, porém, REDO só é aplicado a um item se ele não pertencer a lista

38



Atualização Adiada Multi-usuários

- Desvantagens
 - Limita a concorrência visto que itens permanecem bloqueados até o ponto de efetivação da transação
 - Pode exigir bastante espaço em buffer para controlar itens atualizados até a efetivação da transação

39



Atualização Adiada Multi-usuários

- Vantagens
 - Transação só escreve em disco após efetivada
 - Sem reversão
 - Transações não lêem valores escritos por transações não-efetivadas
 - Sem reversão em cascata

40



Atualização Adiada Multi-usuários

	T_1	T_2	T_3	T_4
(a)	ler_item(A) ler_item(D) escrever_item(D)	ler_item(B) escrever_item(B) ler_item(D) escrever_item(D)	ler_item(A) escrever_item(A) ler_item(C) escrever_item(C)	ler_item(B) escrever_item(B) ler_item(A) escrever_item(A)
(b)	[inicia_transacao, T_1] [escrever_item, T_1, D, 20] [confirma, T_1] [checkpoint] [inicia_transacao, T_2] [escrever_item, T_2, B, 15] [escrever_item, T_2, B, 20] [confirma, T_2] [inicia_transacao, T_3] [escrever_item, T_3, B, 12] [inicia_transacao, T_3] [escrever_item, T_3, A, 30] [escrever_item, T_3, D, 25] ← queda do sistema			

Refeita

Ignoradas

T_2 e T_3 são ignoradas porque não alcançaram seus pontos de efetivação.

T_4 é refeita porque seu ponto de efetivação está depois do último *checkpoint* do sistema.

41



Ações que não afetam BD

- Algumas ações de transações não afetam o BD
 - Gerar relatórios
 - Imprimir mensagens
- Essas ações só deveriam ser feitas após efetivação das transações
- Uma opção é colocá-las como tarefas em lote (*batch jobs*)
- Se a transação for cancelada, a tarefa também é

42



Técnicas Baseadas na Atualização Imediata

- Atualização no BD pode ser imediata, porém apenas após registro no log (*write-ahead log*)
- Temos que reverter efeitos das escritas em disco de transações não efetivadas

43



Técnicas Baseadas na Atualização Imediata

- Duas opções
 - Transações são efetivadas apenas após escritas no BD
 - Algoritmo UNDO / NO-REDO
 - Transações podem ser efetivadas antes das escritas no BD
 - Algoritmo UNDO / REDO
 - Mais genérica
 - Abordada nessa aula
 - Mono-usuário vs multi-usuário

44



Atualização Imediata UNDO / REDO - Mono

- Mudanças no BD das transações ativas devem ser desfeitas
- Algoritmo RIU_S (*Recovery using Immediate Update in a Single-user Environment*)

45



Atualização Imediata UNDO / REDO - Mono

- Algoritmo RIU_S (*Recovery using Immediate Update in a Single-user Environment*)
 - Usa UNDO descrito anteriormente mais REDO
 - Duas listas: efetivadas desde o último checkpoint e ativas
 - UNDO as escritas de itens das transações ativas (ordem inversa do log)
 - REDO escritas de itens das transações efetivadas

46



Atualização Imediata UNDO / REDO - Multi

- Depende dos protocolos de controle de concorrência
- Esboço do protocolo
 - Algoritmo RIU_M (*Recovery using Immediate Update in a Multiuser Environment*)
 - Suposições
 - Log inclui checkpoints
 - Protocolo de concorrência produz planos estritos – p. ex. bloqueio em duas fases estrito

47



Atualização Imediata UNDO / REDO - Multi

- Bloqueio em duas fases estrito
 - Permite que uma transação acesse um item apenas se a transação que escreveu nesse item tenha sido efetivada
 - Deadlocks são possíveis
 - Preciso abortar e desfazer transações

48



Atualização Imediata UNDO / REDO - Multi

- Procedimento RIU_M
 - Duas listas de transações: efetivadas desde o último checkpoint e ativas
 - Desfazer todas as escritas das transações ativas usando UNDO na ordem inversa do log
 - Refazer todas as escritas das transações efetivadas

49



Paginação Shadow

- Não necessita de log em sistemas mono-usuários
- Log apenas em sistemas multi-usuários para controle de concorrência
- BD é composto por número de páginas de tamanho fixo n usadas na recuperação

50



Paginação Shadow

- Catálogo
 - i-ésima entrada aponta para a i-ésima página

51



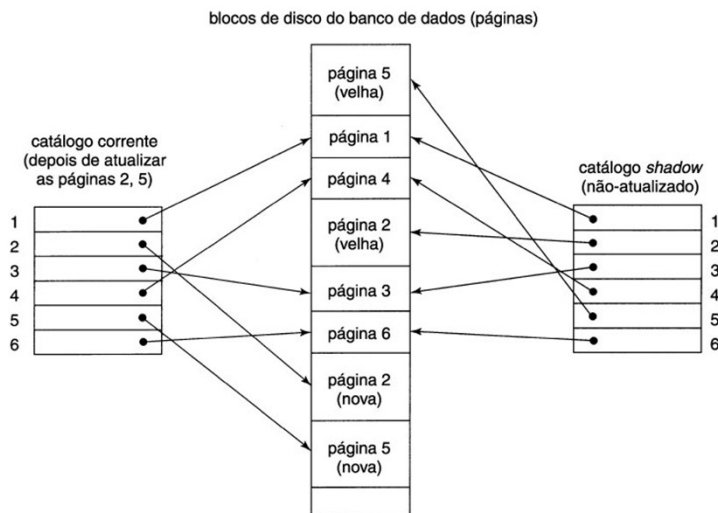
Paginação Shadow

- Início de transação
 - Cria um catálogo *shadow* e grava-o no disco
 - Transação trabalha sobre catálogo atual e nunca altera o *shadow*
 - Escrita de item cria uma cópia atualizada da página que é escrita em outro lugar no disco
 - A entrada do catálogo corrente aponta para a nova página

52



Paginação Shadow



53



Paginação Shadow

- Recuperação
 - Livrar-se das páginas modificadas e descartar o catálogo corrente
 - Estado do BD antes da transação está no catálogo *shadow*
 - Efetivação de uma transação corresponde a descartar o catálogo *shadow*
 - Técnica NO-UNDO / NO-REDO

54



Paginação Shadow

- Ambientes Multi-usuários
 - Incorpora logs e checkpoints
- Desvantagens da técnica
 - Páginas atualizadas mudam de localização em disco sendo difícil manter páginas relacionadas próximas
 - Catálogos grandes geram overhead significativo para escrita de shadow em disco na efetivação

55



Paginação Shadow

- Desvantagens da técnica
 - Manipulação do lixo após efetivação
 - Migração entre páginas deve ser atômica

56



Recuperação em SGBD Múltiplos

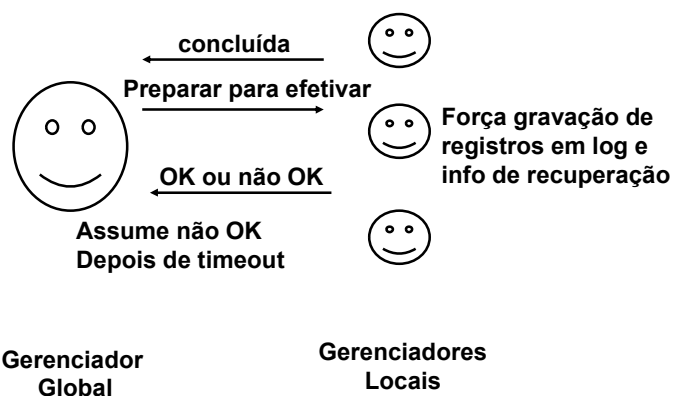
- Transações que acessam múltiplos BDs
 - Localizações diferentes
 - SGBDs diferentes
 - Técnicas de recuperação diferentes
- Mecanismos de recuperação em dois níveis
 - Gerenciador de recuperação global
 - Gerenciadores de recuperação locais

57



Recuperação em SGBD Múltiplos

- Protocolo de efetivação em duas fases
 - FASE 1

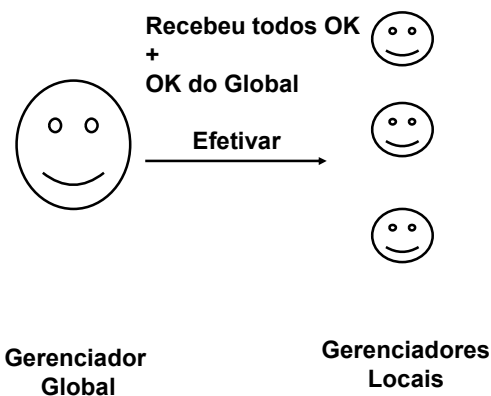


58



Recuperação em SGBD Múltiplos

- Protocolo de efetivação em duas fases
 - FASE 2 (OK)

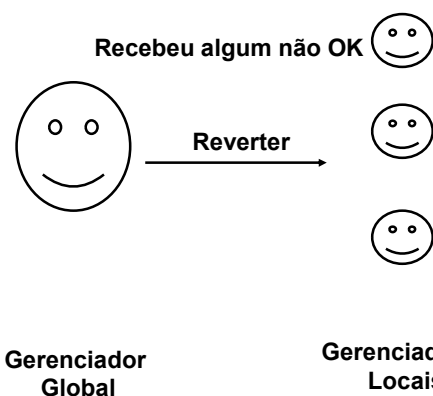


59



Recuperação em SGBD Múltiplos

- Protocolo de efetivação em duas fases
 - FASE 2 (não OK)



60



Backup e Recuperação em Falhas Catastróficas

- Até agora supomos que o log do sistema e páginas *shadow* são mantidos em disco e não são perdidos devido à falha
- E em falhas catastróficas?
 - Backup de todo o banco de dados
 - Dados
 - Logs
 - Log é copiado com mais frequência

61



Resumo

- Técnicas de recuperação de BD
 - Garantir atomicidade da transação
- Esboço informal do processo de recuperação
 - Memória *cache*
 - Atualização local e *shadowing*
 - Imagem anterior e posterior de item
 - Recuperação UNDO e REDO
 - Políticas Roubadas e Não-roubadas

62



Resumo

- Esboço informal do processo de recuperação
 - Políticas forçadas e não-forçadas
 - Confeção de checkpoint do sistema
 - Protocolo para registro adiantado
- Duas abordagens para recuperação
 - Atualização adiada
 - Atualização imediata

63



Resumo

- Técnica de paginação *shadow*
- Recuperação em SGBD múltiplos
 - Efetivação em duas fases
- Recuperação em falhas catastróficas

64



Leitura

- Capítulos 23

65



66